Hardware Debugging

Objectives

After completing this lab, you will be able to:

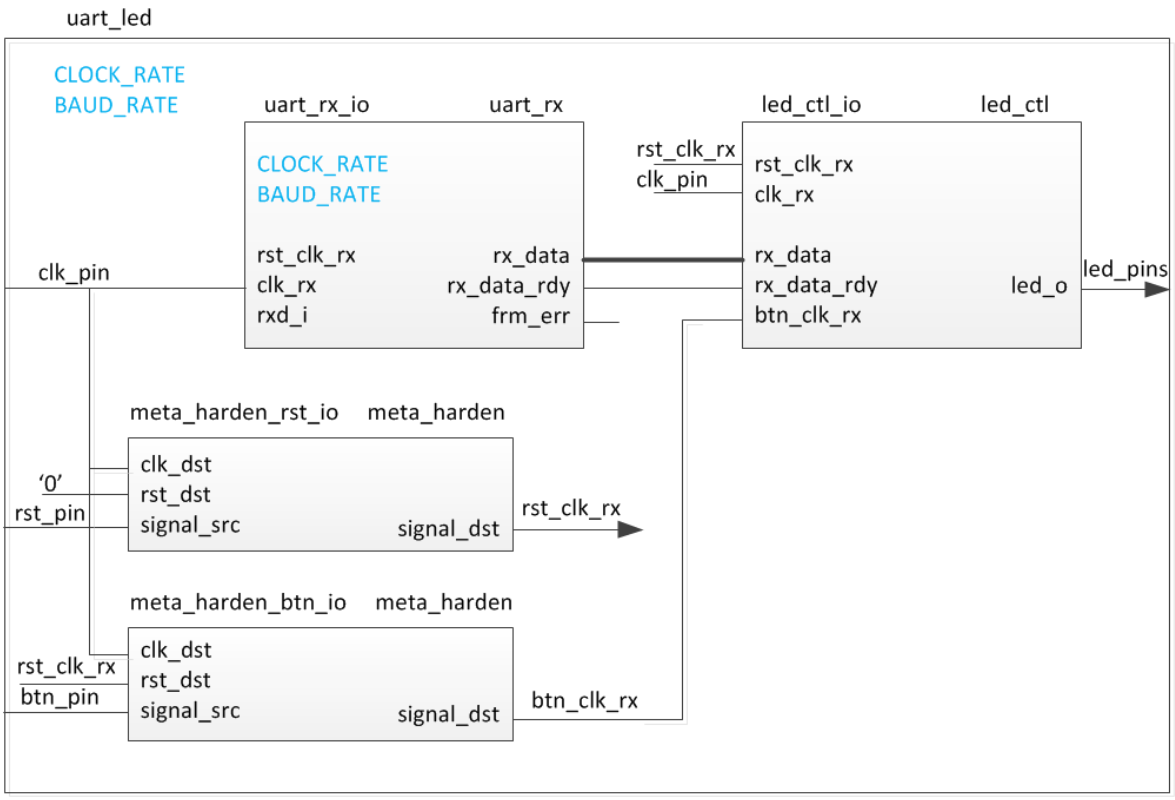
- Use the Integrated Logic Analyzer (ILA) core from the IP Catalog as a debugging tool
- Use Mark Debug feature of Vivado to debug a design
- Use hardware debugger to debug a design

Design Description

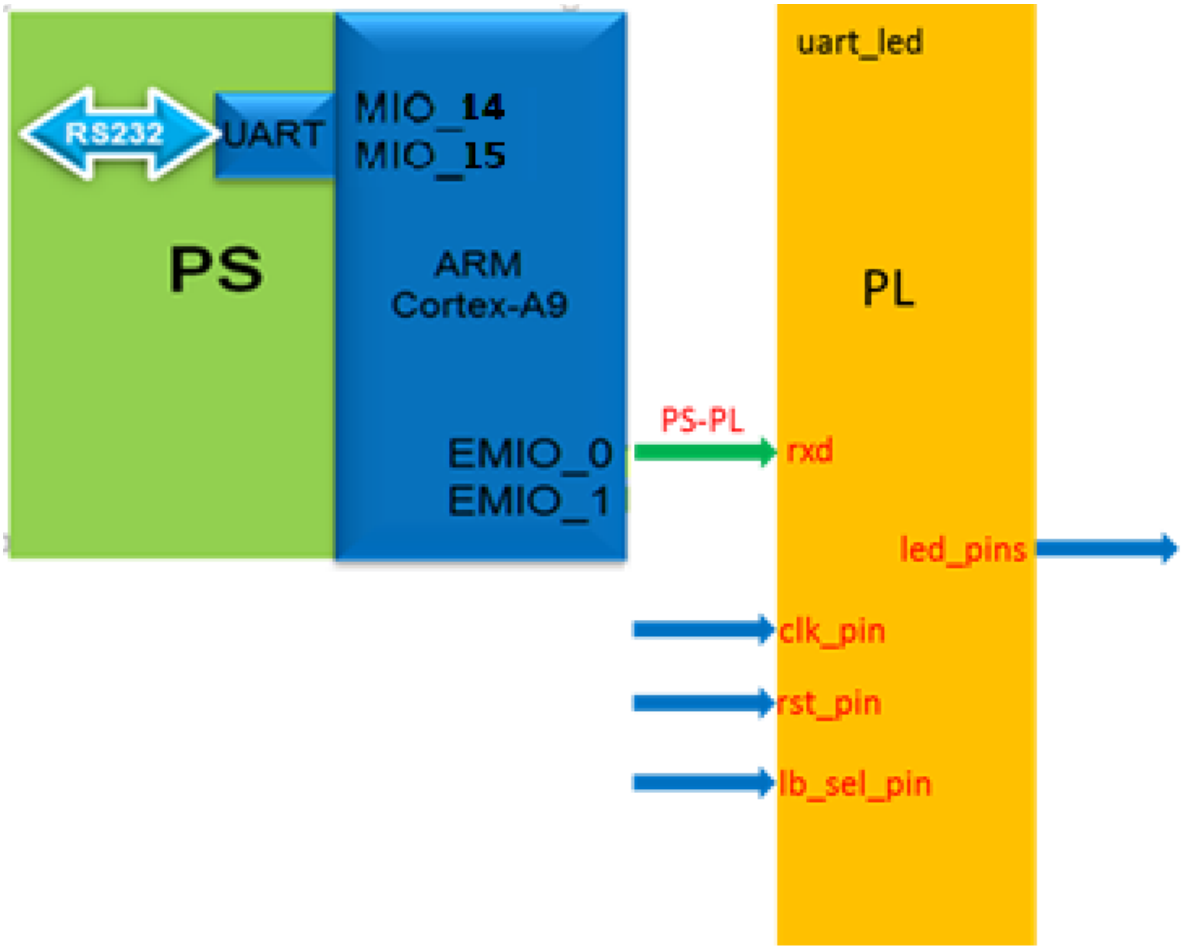
The design consists of a uart receiver receiving the input typed on a keyboard and displaying the binary equivalent of the typed character on the LEDs. When a push button is pressed, the lower and upper nibbles are swapped.

In this design we will use board's USB-UART which is controlled by the Zynq's ARM Cortex-A9 processor. Our PL design needs access to this USB-UART. So first thing we will do is to create a Processing System design which will put the USB-UART connections in a simple GPIO-style and make it available to the PL section.

The provided design places the UART (RX) pin of the PS (Processing System) on the Cortex-A9 in a simple GPIO mode to allow the UART to be connected (passed through) to the Programmable Logic. The processor samples the RX signal and sends it to the EMIO channel 0 which is connected to Rx input of the HDL module provided in the Static directory. This is done through a software application provided in the lab6.sdk folder hierarchy.



The Complete Design on PL



The Complete System

Steps

Create a Vivado Project using IDE

In this design we will use board's USB-UART which is controlled by the Zynq's ARM Cortex-A9 processor. Our PL design needs access to this USB-UART. So first thing we will do is to create a Processing System design which will put the USB-UART connections in a simple GPIO-style and make it available to the PL section.

Launch Vivado and create a project targeting the XC7Z020clg400-1 device, and use provided the tcl script file (ps7_create_pynq.tcl) to generate the block design for the PS subsystem. Also, add the Verilog HDL files, uart_led_pins_pynq.xdc and uart_led_timing_pynq.xdc files from the *<2018_2_zynq_sources>\lab6* directory.

<2018_2_zynq_labs> refers to the **C:\xup\fpga_flow\2018_2_zynq_labs** directory and **<2018_2_zynq_sources>** refers to the **C:\xup\fpga_flow\2018_2_zynq_sources** directory.

1. Open Vivado by selecting **Start > Xilinx Design Tools > Vivado 2018.2**

2. Click **Create New Project** to start the wizard. You will see *Create A New Vivado Project* dialog box. Click **Next**.
3. Click the *Browse* button of the *Project location* field of the **New Project** form, browse to **<2018_2_zynq_labs>**, and click **Select**.
4. Enter **lab6** in the *Project name* field. Make sure that the *Create Project Subdirectory* box is checked. Click **Next**.
5. Select **RTL Project** option in the *Project Type* form, and click **Next**.
6. Using the drop-down buttons, select **Verilog** as the *Target Language* and *Simulator Language* in the *Add Sources* form.
7. Click on the **Blue Plus** button, then the **Add Files...** button and browse to the **<2018_2_zynq_sources>\lab6** directory, select all the Verilog files *(led_ctl.v, meta_harden.v, uart_baud_gen.v, uart_led.v, uart_rx.v, uart_rx_ctl.v and uart_top.v),* click **OK**, and then click **Next**.
8. Click **Next** to get to the *Add Cons*traints form.
9. Click on the **Blue Plus** button, then **Add Files...** and browse to the **c:\xup\fpga_flow\2018_2_zynq_sources\lab6** directory (if necessary), select *uart_led_timing_pynq.xdc* and the appropriate *uart_led_pins_pynq.xdc* and click **Open**.
10. Click **Next.**
11. In the *Default Part* form, Use the **Boards** option, you may select the **PYNQ-Z1** or the **PYNQ-Z2** depending on your board from the Display Name drop down field.

    You may also use the **Parts** option and various drop-down fields of the **Filter** section, select the **XC7Z020clg400-1** part.

    Notice that PYNQ-Z1 and PYNQ-Z2 may not be listed under the Boards menu as they are not in the tools database. If not listed then you can download the board files for the desired boards.
12. Click **Next**.
13. Click **Finish** to create the Vivado project.
14. In the Tcl Shell window enter the following command to change to the lab directory and hit the Enter key.

    cd C:/xup/fpga_flow/2018_2_zynq_sources/lab6
15. Generate the PS design by executing the provided Tcl script.

    source ps7_create_pynq.tcl

    This script will create a block design called *system*, instantiate ZYNQ PS with one GPIO channel (GPIO14) and one EMIO channel. It will then create a top-level wrapper file called system_wrapper.v which will instantiate the system.bd (the block design). You can check the contents of the tcl files to confirm the commands that are being run.
16. Double-click on the **uart_led** entry to view its content.

    Notice in the Verilog code, the BAUD_RATE and CLOCK_RATE parameters are defined to be 115200 and 125 MHz respectively.

```
22  //
23
24  `timescale 1ns/1ps
25  module uart_led (
26    // Write side inputs
27    input             clk_pin,       // Clock input (from pin)
28    input             rst_pin,       // Active HIGH reset (from pin)
29    input             btn_pin,       // Button to swap high and low bits
30    input             rxd_pin,       // RS232 RXD pin - directly from pin
31    output     [7:0]  led_pins,       // 8 LED outputs
32    output            rx_data_rdy_out
33  );
34
35  //*****************************************************************************
36  // Parameter definitions
37  //*****************************************************************************
38    parameter BAUD_RATE            = 115_200;
39    parameter CLOCK_RATE           = 125_000_000;
40
41  //*****************************************************************************
42  // Reg declarations
43  //*****************************************************************************
44
```

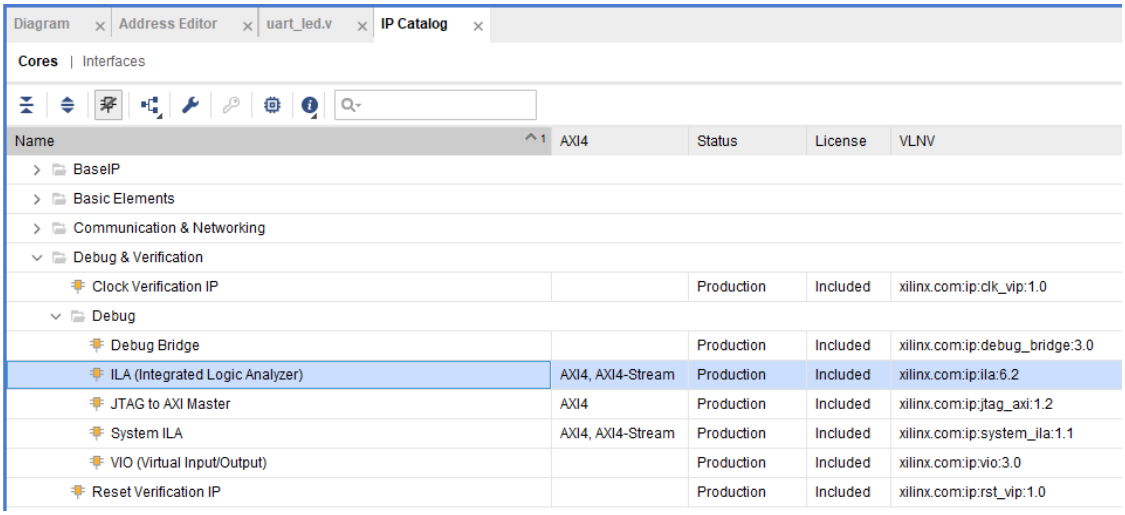
CLOCK_RATE parameter of uart_led

Add the ILA Core

1. Click **Flow Navigator > PROJECT MANAGER > IP Catalog**.

   The catalog will be displayed in the Auxiliary pane.

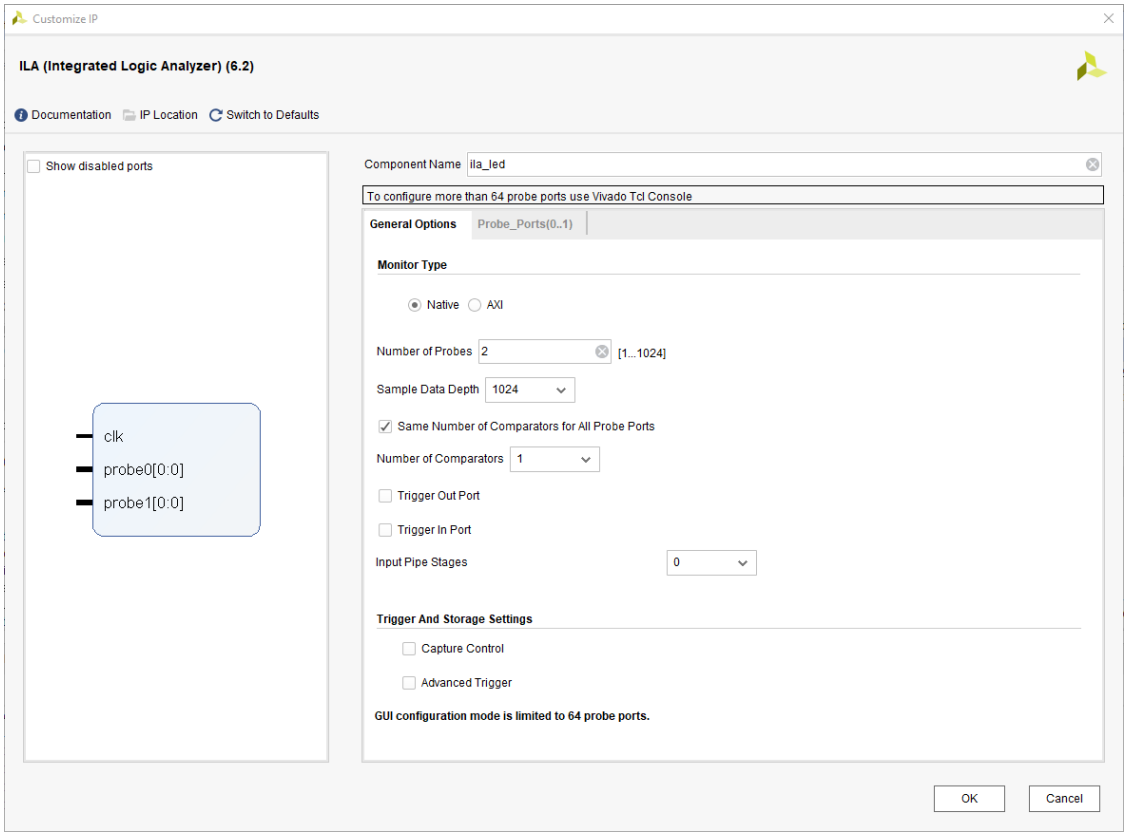
2. Expand the **Debug & Verification > Debug** folders and double-click the **ILA** entry.



ILA in IP Catalog

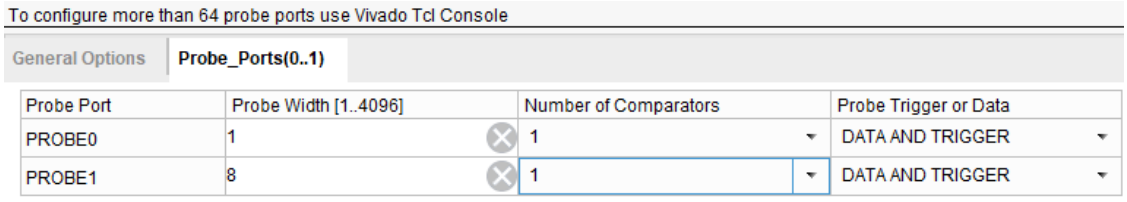
This exercise will be connecting the ILA core/component to the LED port which is 8-bit wide.

3. Click **Customize IP** on the following Add IP window. The ILA IP will open.
4. Change the component name to **ila_led**.
5. Change the *Number of Probes* to **2**.



Setting the component name and the number of probes field

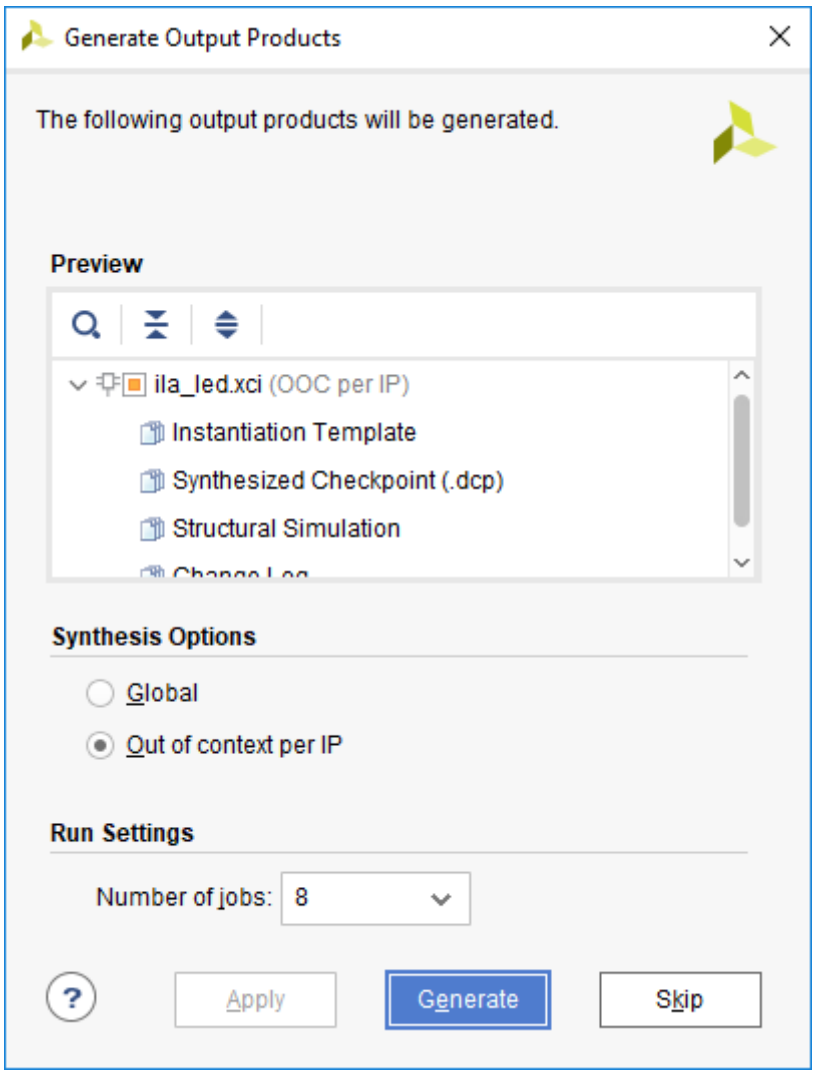
6. Select the *Probe Ports* tab and change the PROBE1 port width to **8**, leaving the PROBE0 width to **1**.



Setting the probes widths

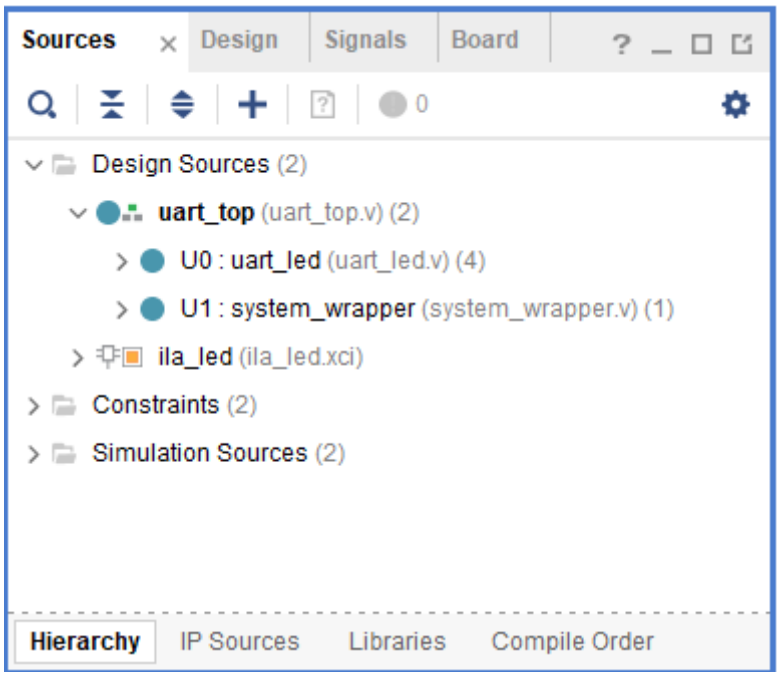
7. Click **OK**.

   The Generate Output Products dialog box will appear.



The Generate Output Products

8. Click the **Generate** button to generate the core including the instantiation template. Click **OK** at the warning box. Notice the core is added to the *Design Sources* view.



Newly generate ila core added in the design source

9. Select the **IP Sources** tab, expand the **IP(1) > ila_led > Instantiation Template**, and double-click the **ila_led.veo** entry to see the instantiation template.
10. Instantiate the ila_led in design by copying lines 56 – 62 and pasting to ~line 69 (before "endmodule" on the last line) in the uart_top.v file.
11. Change *your_instance_name* to **ila_led_i0.**
12. Change the following port names in the Verilog code to connect the ila to existing signals in the design:

    .clk(CLK) . clk(clk_pin)

    .probe0(PROBE0) . probe0(rx_data_rdy_out)

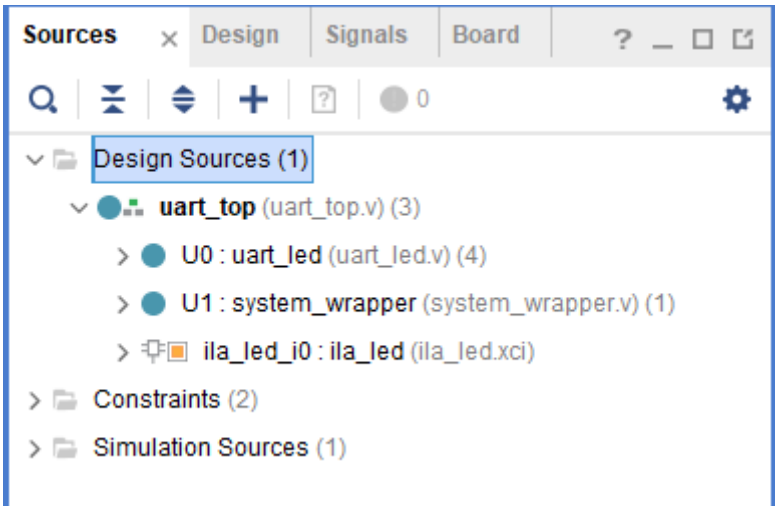
    .probe1(PROBE1) . probe1(led_pins)

```
69      ila_led ila_led_i0 (
70          .clk(clk_pin), // input wire clk
71          .probe0(rx_data_rdy_out), // input wire [0:0]  probe0
72          .probe1(led_pins) // input wire [7:0]  probe1
73      );
74  endmodule
```

Instantiating the ILA Core in the uart_top.v

13. Select **File > Save File**.

    Notice that the ILA Core instance is in the design hierarchy.



ILA Core added to the design

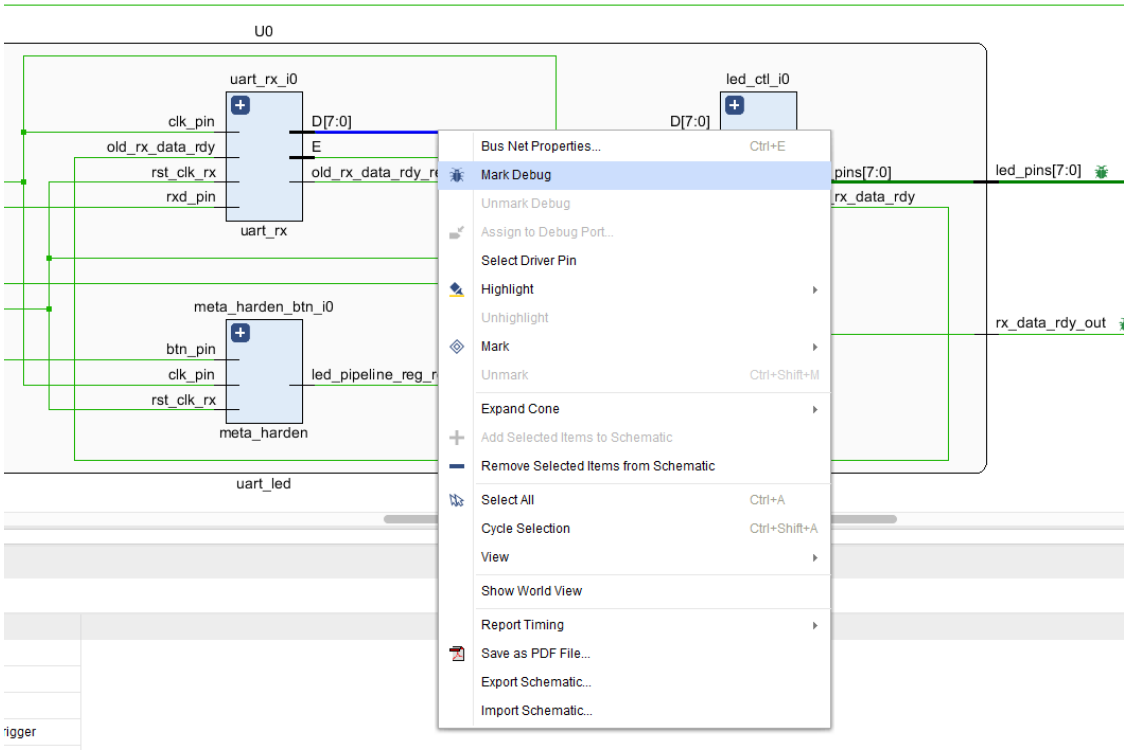
Synthesize the Design and Mark Debug

Synthesize the design. Open the synthesized design. View the schematic. Add Mark Debug on the rx_data bus between the uart_rx_i0 and led_ctl_i0 instances.

1. Click **Flow Navigator > SYNTHESIS > Run Synthesis**. Click **Save** to Save Project if prompted.

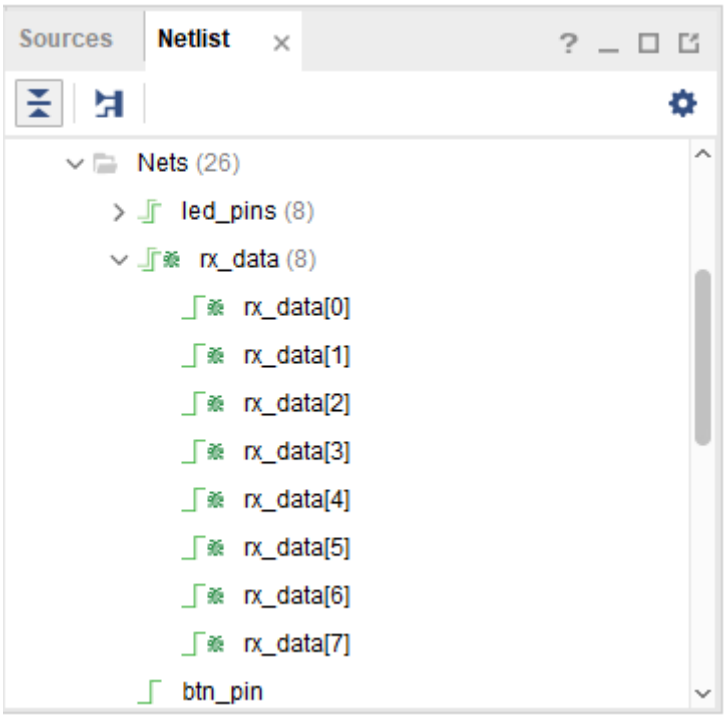
   The synthesis process will be run on the uart_top.v and all its hierarchical files. When the process is completed a *Synthesis Completed* dialog box with three options will be displayed.

2. Select the *Open Synthesized Design* option and click **OK**.
3. Click on **Flow Navigator > SYNTHESIS > Synthesized Desig > Schematic** to view the synthesized design in a schematic view.
4. Expand component **U0** and Select the **rx_data** bus between the *uart_rx_i0* and the *led_ctl_i0* instances, right-click, and select **Mark Debug**.



Marking a bus to debug

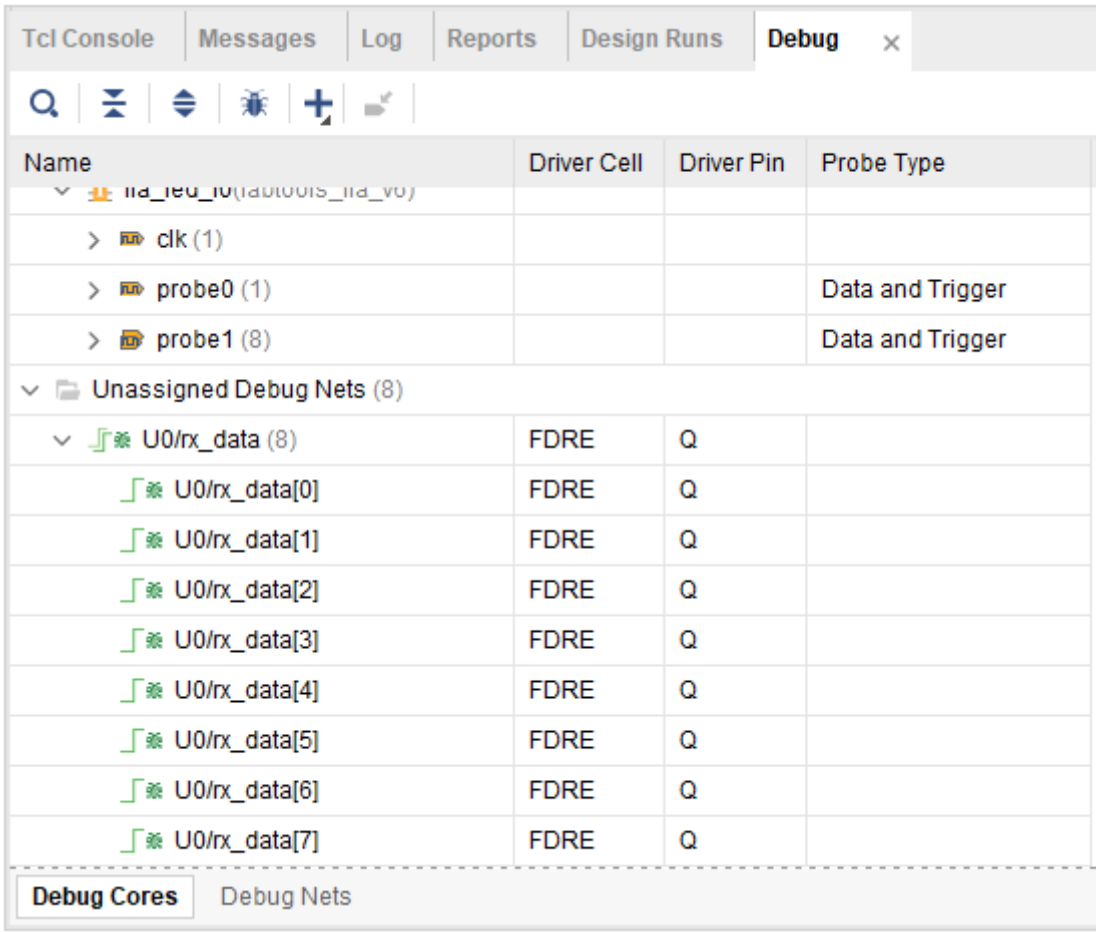
5. Select **File > Constraints > Save**.
6. Click **OK**, and then again **OK** to use **uart_led_timing_pynq.xdc** as the target.
7. Select the **Netlist** tab and notice that the nets which are marked/assigned for debugging have a debug icon next to them.



Nets with debug icons

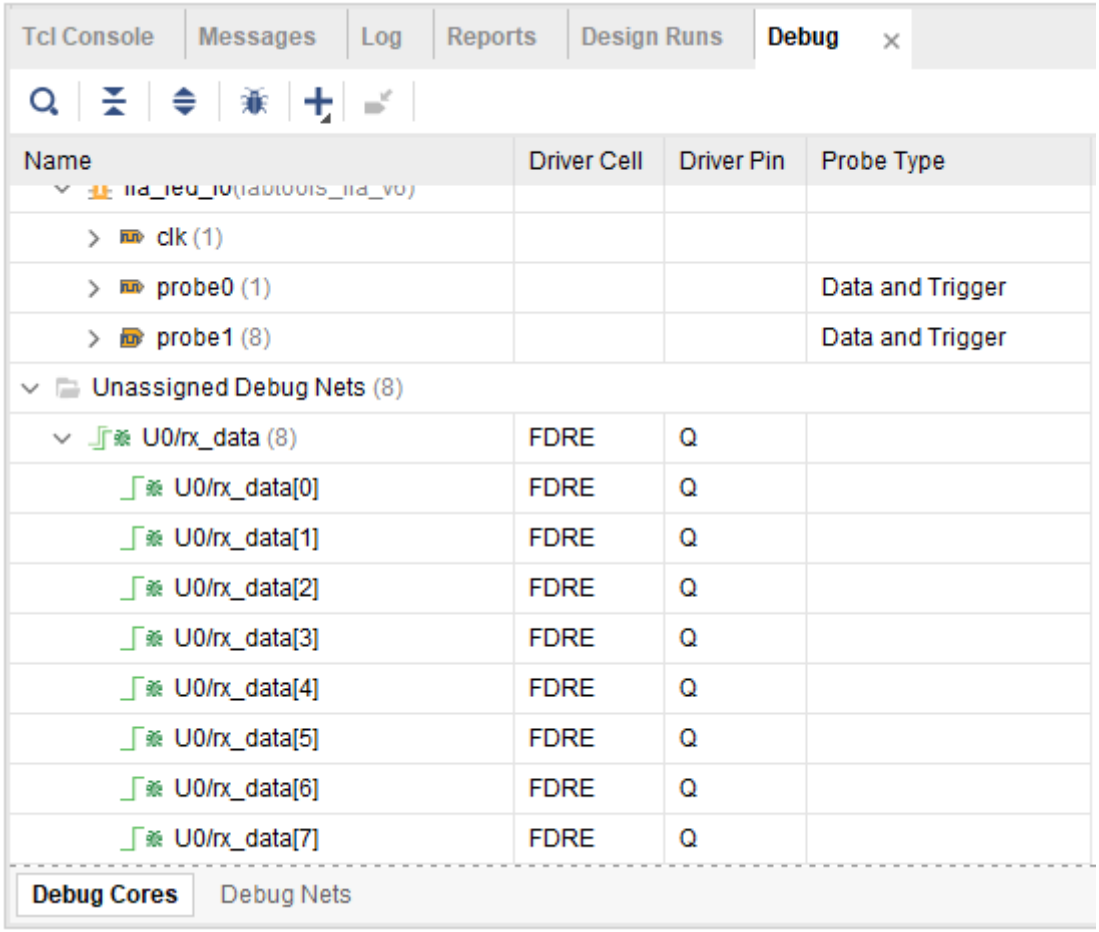
8. Select the **Debug** layout or **Layout > Debug**.

   Notice that the **Debug** tab is visible in the Console pane showing Assigned and Unassigned Debug Nets groups.



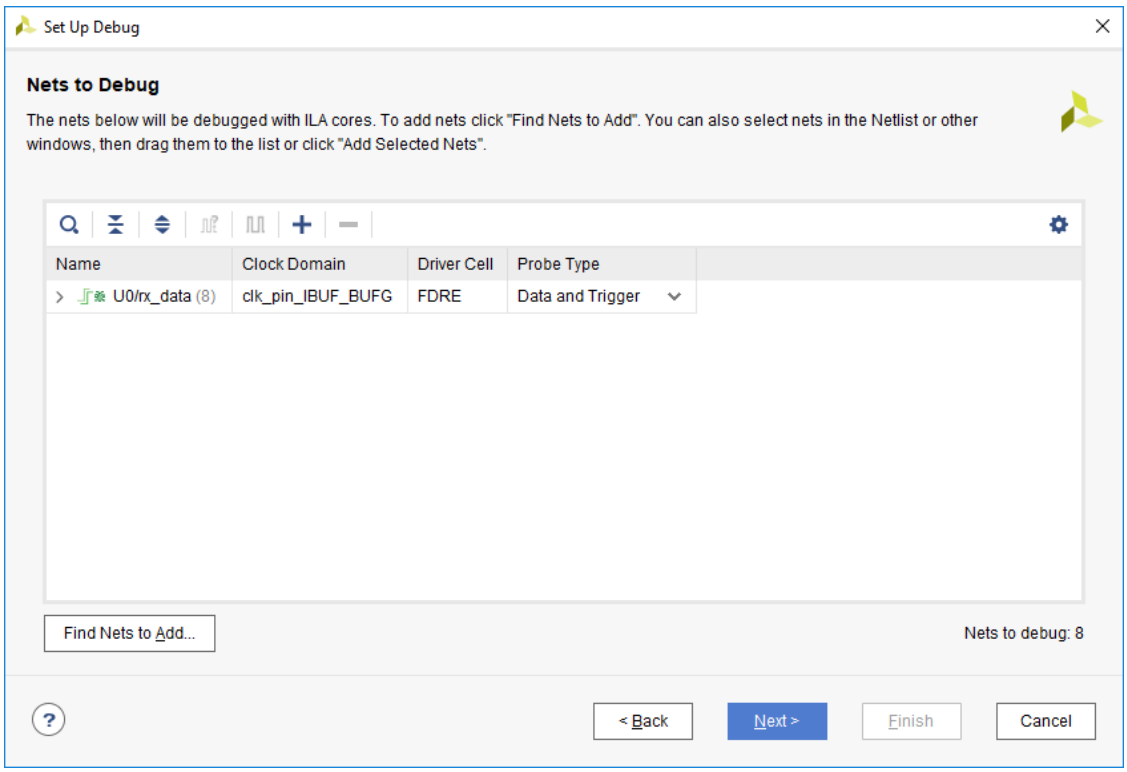
Debug tab showing assigned and unassigned nets

9. Either click on the  button in the top vertical tool buttons of the Debug pane, or right-click on the *Unassigned Debug Nets* and select the **Set up Debug...** option.



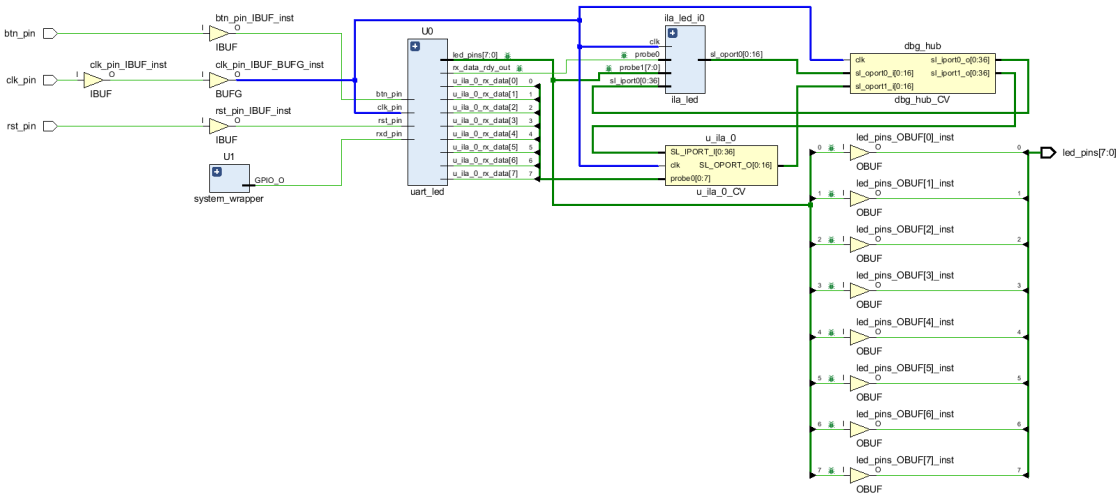
10. In the *Set Up Debug* wizard click **Next**.

    Note that rx_data is listed, with the Clock Domain as clk_pin_IBUF_BUFG.



The remaining nets after removing already assigned nets in the Set Up Debug wizard

11. Click **Next** and again **Next** (leaving everything as defaults) then **Finish**.
12. In the Synthesized Design Schematic, click on the net on the output side of the BUFG for the input pin named clk_pin. Hover over the now highlighted net and notice the name is clk_pin_IBUF_BUFG. This is the clock net selected for the debug nets earlier.



Locating clk_pin_IBUF_BUFG in the design

13. Right click on **uart_led_pins_pynq.xdc** in the sources pane and select **Set as Target Constraint File**. This will save the changes to the file
14. Select **File > Constraints > Save** and click **OK** and Click **Yes.**

15. Open *uart_led_pins_pynq.xdc* and notice the debug nets have been appended to the bottom of the file.
16. Perform this step if synthesis is not already up-to-date: In the **Design Runs** tab, right-click on the *synth_1* and select **Force Up-to-Date** to ensure that the synthesis process is not re-run, since the design was not changed.

Implement and Generate Bitstream

Generate the bitstream.

1. Click on the **Generate Bitstream** to run the implementation and bit generation processes.
2. Click **Yes** to run the implementation processes.
3. When the bitstream generation process has completed successfully, a box with three options will appear. Select the **Open Hardware Manager** option and click **OK**.

Debug in Hardware

Connect the board and power it ON. Open a hardware session, and program the FPGA.

1. Make sure that the Micro-USB cable is connected to the JTAG PROG connector.
2. Turn ON the power.
3. Select the *Open Hardware Manager* option and click **OK**.

   The Hardware Manager window will open indicating “unconnected” status.
4. Click on the **Open target** link, then **Auto Connect** from the dropdown menu.

   You can also click on the **Open recent target** link if the board was already targeted before.
5. The Hardware Session status changes from Unconnected to the server name and the device is highlighted. Also notice that the Status indicates that it is not programmed.
6. Select the device and verify that the **uart_top.bit** is selected as the programming file in the General tab. Also notice that there is an entry in the *Debug probes file* field.

Start a terminal emulator program such as TeraTerm or HyperTerminal. Select an appropriate COM port (you can find the correct COM number using the Control Panel). Set the COM port for 115200 baud rate communication. Program the FPGA and verify the functionality.

1. Start a terminal emulator program such as TeraTerm or HyperTerminal.
2. Select an appropriate COM port (you can find the correct COM number using the Control Panel).
3. Set the COM port for 115200 baud rate communication.
4. Right-click on the FPGA, and select **Program Device...** and click **Program**.

   The programming bit file be downloaded and the DONE light will be turned ON indicating the FPGA has been programmed. Debug Probes window will also be opened, if not, then select **Window > Debug Probes.**

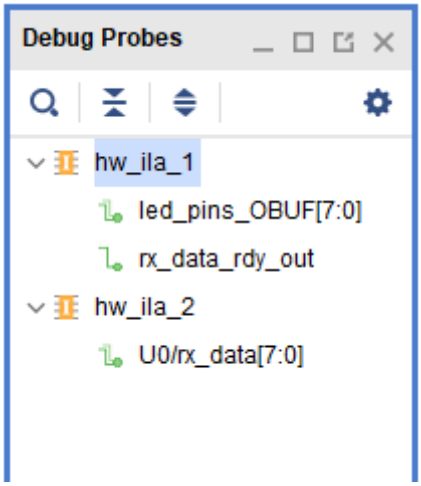
Start a SDK session, point it to the c:\xup\fpga_flow\2018_2_zynq_sources\lab6\pynq\lab6.sdk workspace

1. Open **SDK** by selecting **Start > Xilinx Design Tools > Xilinx SDK 2018.2**

2. In the **Select a workspace** window, click on the browse button, browse to C:\xup\fpga_flow\2018_2_zynq_sources\lab6\pynq\lab6.sdk and click **OK**.
3. Click **OK**.

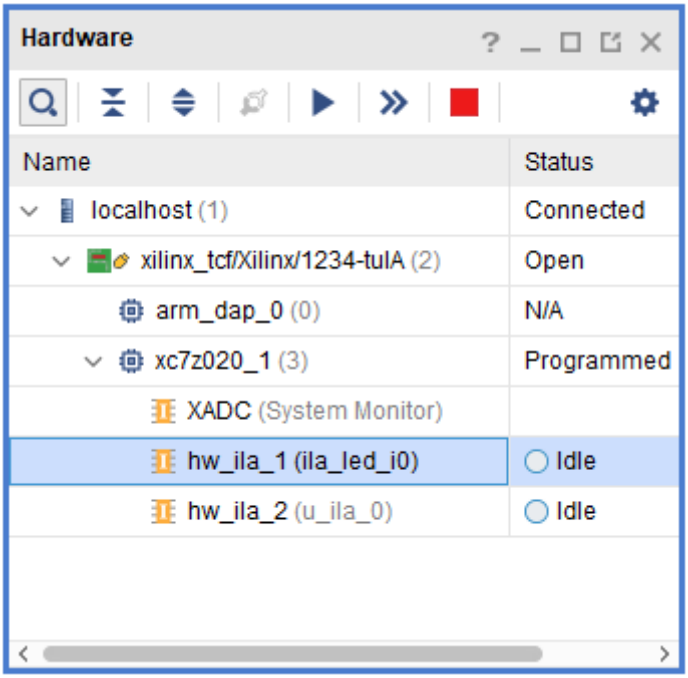
   In the *Project Explorer*, right-click on the uart_led_zynq, select *Run As*, and then **Launch on Hardware (System Debugger)**

   In the Hardware window in Vivado notice that there are two debug cores, hw_ila_1 and hw_ila_2.



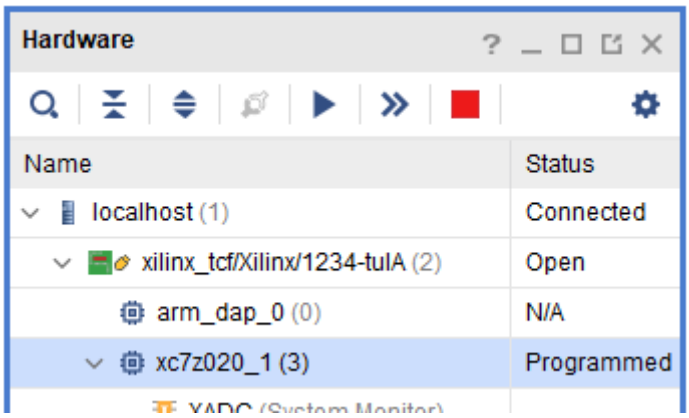
Debug probes

The hardware session status window also opens showing that the FPGA is programmed having two ila cores with the idle state.



Hardware session status

Select the target FPGA xc7z020_1), and click on the **Run Trigger Immediate** button to see the signals in the waveform window.

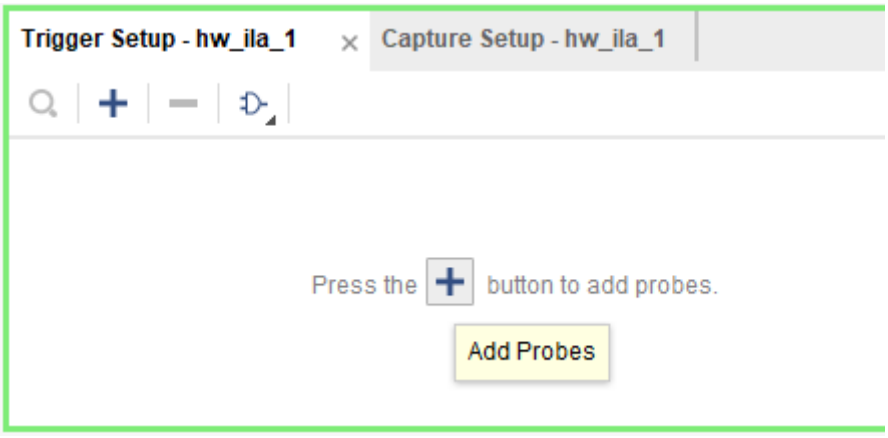


Opening the waveform window

Two waveform windows will be created, one for each ila; one ila is of the instantiated ILA core and another for the MARK DEBUG method.

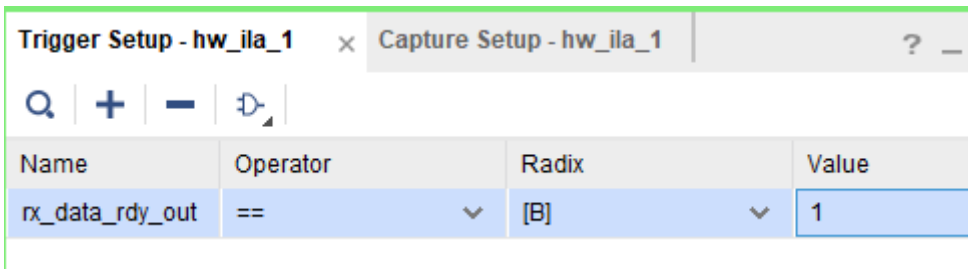
Setup trigger conditions to trigger on a write to led port (rx_data_rdy_out=1) and the trigger position to 512. Arm the trigger.

1. In the **Trigger Setup** window, click *Add Probes* and select the **rx_data_rdy_out**.



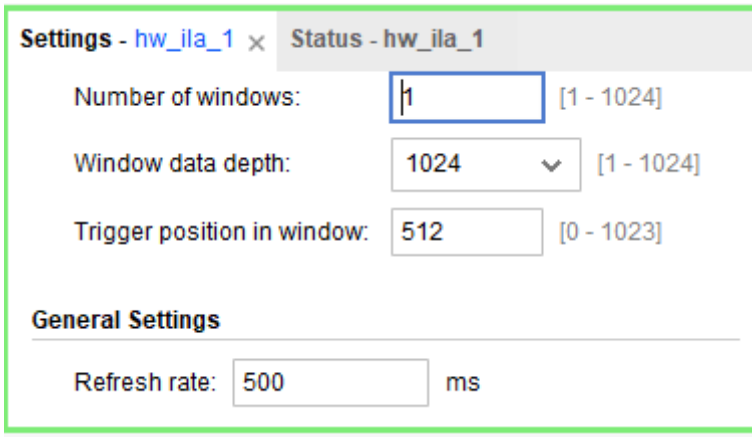
Adding a signal to trigger setup

2. Set the compare value (== [B] X) and change the value from x to 1. Click **OK**.



Setting the trigger condition

3. Set the trigger position of the *hw_ila_1* to **512**.

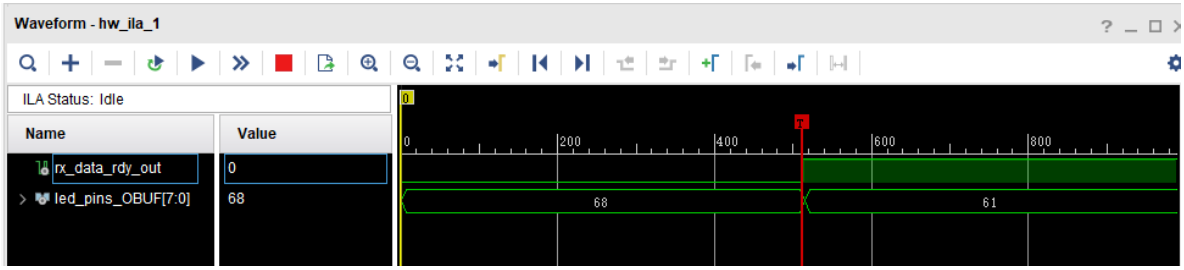


Setting up the trigger position

4. Similarly, set the trigger position of the *hw_ila_2* to **512**.
5. Select the *hw_ila_1* in the Hardware window and then click on the Run Trigger (▶) button. Observe that the hw_ila_1 core is armed and showing the status as **Waiting for Trigger**.

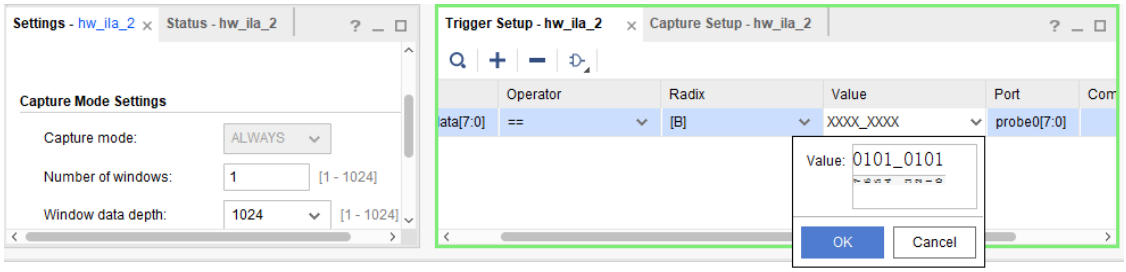
Hardware analyzer running in capture mode

6. In the terminal emulator window, type a character, and observe that the hw_ila_1 status changes from capturing to idle as the rx_data_rdy_out became 1.
7. Select the hw_ila_data_1.wcfg window and see the waveform. Notice that the *rx_data_rdy_out* goes from 0 to 1 at 512th sample.



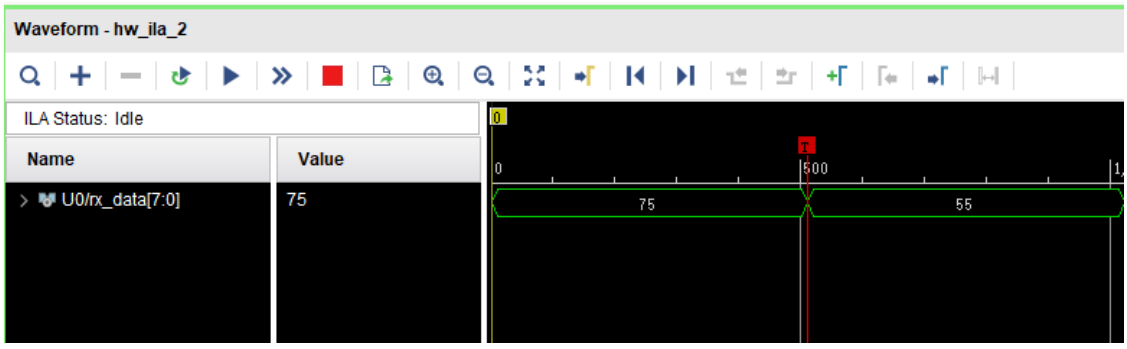
Zoomed waveform view

8. Add the hw_ila_2 probes to the trigger window of the hw_ila_2 and change the trigger condtion for rx_data[7:0]'s Radix from Hexadecimal to Binary. Change XXXX_XXXX to **0101_0101** (for the ASCII equivalent of U).



Setting up trigger condition for a particular input pattern

9. In the Hardware window, right-click on the **hw_ila_2** and select **Run Trigger,** and notice that the status of the hw_ila_2 changes from *idle* to *Waiting for Trigger.* Also notice that the hw_ila_1 status does not change from idle as it is not armed.
10. Switch to the terminal emulator window and type **U** (shift+u) to trigger the core.
11. Select the corresponding waveform window and verify that it shows 55 after the trigger.



Second ila core triggered

12. When satisfied, close the terminal emulator program and power OFF the board.
13. Select **File > Close Hardware Manager**. Click **OK** to close it.
14. Close the **Vivado** program by selecting **File > Exit** and click **OK**.
15. Close the **SDK** program by selecting **File > Exit** and click **OK**.

Conclusion

You used ILA core from the IP Catalog and Mark Debug feature of Vivado to debug the hardware design.